

Relatório 4: Método de Convolução em Processamento de Imagem - Desfoque Gaussiano

Jonatan Felipe Hartmann

Universidade Federal de Santa Catarina
Ciências da Computação
Florianópolis, Brasil

Lucas de Souza Vieira

Universidade Federal de Santa Catarina
Ciências da Computação
Florianópolis, Brasil

Lucas Tomio Schwachow

Universidade Federal de Santa Catarina
Ciências da Computação
Florianópolis, Brasil

Pedro Henrique Gimenez

Universidade Federal de Santa Catarina
Ciências da Computação
Florianópolis, Brasil

Rodrigo Schwartz

Universidade Federal de Santa Catarina
Ciências da Computação
Florianópolis, Brasil

Victoria Rodrigues Veloso

Universidade Federal de Santa Catarina
Ciências da Computação
Florianópolis, Brasil

RESUMO

Este relatório apresenta o desenvolvimento de um sistema baseado em VHDL utilizando Quartus II para processamento de imagens através da aplicação de um filtro de Desfoque Gaussiano. O projeto se concentra na implementação de uma operação de convolução entre a matriz do kernel do filtro e a matriz da imagem para obter o efeito de desfoque. A arquitetura VHDL executa a convolução em matrizes 3x3, gerando uma matriz modificada que representa a imagem borrada. O sistema desenvolvido visa aprimorar a compreensão e implementação de algoritmos de processamento de imagens dentro de uma estrutura de linguagem de descrição de hardware (VHDL).

O fluxo de trabalho começa com a importação de uma representação de imagem baseada em matriz, convertida anteriormente por um script Python, para o ambiente VHDL, seguida pela aplicação do algoritmo de Desfoque Gaussiano por meio de operações de convolução. O processo de convolução envolve multiplicação e soma de pixels, realizada iterativamente na matriz da imagem e na matriz do filtro. Após a conclusão da convolução, o sistema produz uma matriz modificada que captura o efeito desfocado.

Além disso, um script Python é utilizado para transformar a matriz resultante novamente em uma imagem JPG, permitindo a representação visual e a avaliação do desfoque aplicado.

Palavras-chave—VHDL, processamento de imagem, filtro de desfoque gaussiano, matriz de convolução, implementação de hardware, Quartus II, transformação de imagem

I. INTRODUÇÃO

A convolução é uma importante operação matemática utilizada no processamento de sinais. Seu fundamento consiste em combinar duas funções, gerando uma terceira que representa a forma como uma modifica a outra. A figura 1 mostra a expressão geral da convolução aplicada no processamento de imagens.

$$g(x, y) = \omega * f(x, y) = \sum_{i=-a}^a \sum_{j=-b}^b \omega(i, j) f(x - i, y - j),$$

Fig. 1. Expressão de convolução para aplicação de filtros em imagens
Fonte: wikipedia, 2023

O Desfoque Gaussiano é um filtro comumente utilizado para redução de irregularidades e suavização do valor do pixel de uma fotografia. Entre as principais vantagens de usar o Desfoque Gaussiano, destacam-se a facilidade de visualização e a eficiência em reduzir ruídos para suavizar uma imagem. Este filtro apresenta uma matriz, também conhecida como kernel, que está ilustrada na imagem 2.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Fig. 2. Kernel para o filtro gaussiano
Fonte: wikipedia, 2023

Com isso, o presente trabalho tem como objetivo demonstrar a aplicação do filtro de desfoque Gaussiano em imagens, utilizando uma arquitetura que aplica o método de convolução, feita na linguagem de descrição de hardware VHDL. No documento, será apresentado como foi organizado o script Python para transformar uma imagem em uma matriz e como foram estruturados os componentes do sistema digital de aplicação de convolução entre kernel do filtro e imagem

II. METODOLOGIA

A. Script Python

O script de conversão da imagem em uma matriz foi realizado na linguagem Python. Além da conversão mencionada, também foi utilizado para realizar a convolução, em que seu resultado será utilizado posteriormente no

testbench. Também foi codificada a função que transforma a matriz final em imagem de novo para a representação visual do resultado.

1) **Conversor de Imagem para Matriz:** O script utiliza a biblioteca *Pillow* que gerencia tratamentos de imagens. Inicia-se lendo a imagem recebida pela função. Após a leitura, a imagem é convertida para tons na escala cinza para facilitar o cálculo posterior, e é redimensionada para o tamanho desejado. O tamanho padrão da imagem final é 600x600 pois o sistema digital em VHDL será baseado neste tamanho. Após o redimensionamento, o valor de cada pixel em decimal é obtido. A faixa de valores dos pixels é de [0, 255], visto que a imagem possui apenas tons da escala de cinza, sendo 0 preto e 255 branco. Depois, é realizada a conversão dos valores de cada pixel, que estavam em decimais, para binários de 8 bits, e então, a matriz é criada. Com a matriz criada, é realizado um tratamento, onde os valores extremos da matriz (primeira linha, última linha, primeira coluna e última coluna) são copiados e inseridos ao redor da matriz, redimensionando-a para uma matriz 602x602. Esse tratamento visa evitar o acesso de posições não existentes no momento de aplicação do filtro e mantém um valor aproximado para as bordas da imagem. Por fim, é feita a leitura da matriz, percorrendo todas as colunas de cada linha, e salvando esses valores em um arquivo .txt, onde cada linha desse arquivo representa o valor em binário de 8 bits de um pixel da matriz da imagem. Assim, posteriormente, esse arquivo será utilizado como o arquivo de estímulos para o testbench do sistema de convolução descrito em VHDL.

2) **Aplicação da Convolução:** Para simular o que irá ser realizado em hardware, foi realizado um script em Python com uma estrutura próxima a de uma na linguagem VHDL, utilizando sinais e *while loops*. A função recebe como parâmetro o arquivo .txt da imagem convertida em matriz e o arquivo .txt do *kernel* em binário. O script começa salvando os dados recebidos em listas, uma para a imagem e outra para o *kernel*, em que seus valores são convertidos para decimal para o facilitamento do cálculo em Python. Em seguida, cria-se uma lista simulando a memória de saída em que todos os valores são "255" para serem posteriormente alterados durante o processamento. Começa-se um *while loop* que percorre as linhas da matriz e outro *while loop* dentro percorrendo cada coluna. Dentro de cada coluna, começa o *while loop* percorrendo a matriz 3x3 do *kernel* e do "corte" da imagem. Calcula-se a convolução de cada pixel somando os produtos da matriz com o kernel. Em seguida, o sistema armazena o valor resultante na posição desejada da matriz final. Após realizar as operações, a função salva a matriz em um arquivo "matrizfiltrada.txt" em que cada linha é um pixel. O arquivo final possui 600x600 linhas.

3) **Conversor de Matriz para Imagem:** Para visualizar o resultado final, foi criada uma função que recebe um arquivo .txt com a matriz e salva uma imagem .jpg do resultado. A função começa chamando uma outra função que realiza

a leitura de um arquivo .txt e transforma seu conteúdo em uma matriz. Com a matriz extraída do arquivo, começa-se a percorrer cada elemento da lista e o transforma em um pixel por meio da biblioteca *Pillow*. Após cada pixel ser armazenado, a função salva a imagem final em um arquivo .jpg.

B. Projeto RTL

O desenvolvimento do sistema e dos componentes utilizados tiveram como base o pseudo algoritmo que está ilustrado em II-B, visando realizar as mesmas etapas e operações feitas nele para a aplicação da convolução, mas no contexto da linguagem VHDL. Conforme o circuito foi sendo modelado, algumas modificações foram realizadas, ou seja, o sistema digital final teve diversas mudanças em relação ao pseudocódigo base. O diagrama de blocos do projeto com as modificações pode ser visualizado nas referências [2]

Algoritmo para Processamento de Imagens

```

linha ← 1; coluna ← 1; pronto ← 0; memsaida ← [vazio]; mem ← data; kernel ← data;
{Carregar memória e kernel com o arquivo}
while linha < 600 do
  while coluna < 600 do
    soma ← 0; i ← 0; deslocaLinha ← -1; deslocaColuna ← -1;
    while i < 9 do
      soma ← soma + (kernel[i] * mem[(linha + deslocaLinha)*602 + (coluna + deslocaColuna)]);

      soma ← soma << 4 {DIVIDINDO POR 16 UNICAMENTE NO FILTRO GAUSSIANO}
      i ← i + 1; deslocaColuna ← deslocaColuna + 1; memsaida[linha * 602 + coluna] ← soma;
      if deslocaColuna > 1 then
        deslocaColuna ← 0; deslocaLinha ← deslocaLinha + 1;
      end if
    end while
    pronto ← 1;
  end while
end while
{Acessar memória: mem[linha*602 + coluna]}
{Memórias acessadas em cada iteração do kernel:}
mem[(linha - 1) * 602 + (coluna - 1)]
mem[(linha - 1) * 602 + (coluna)]
mem[(linha - 1) * 602 + (coluna + 1)]
mem[(linha) * 602 + (coluna - 1)]
mem[(linha) * 602 + (coluna)]
mem[(linha) * 602 + (coluna + 1)]
mem[(linha + 1) * 602 + (coluna - 1)]
mem[(linha + 1) * 602 + (coluna)]
mem[(linha + 1) * 602 + (coluna + 1)]
{Codificação de cada "variável" do algoritmo:}
linha : STD_LOGIC_VECTOR(8 downto 0)
coluna : STD_LOGIC_VECTOR(8 downto 0)
pronto : STD_LOGIC

```


continuam sendo habilitados, até que que "fim i" seja positivo)

3) **Testbench:** Dentro do testbench, as três memórias do sistema foram inicializadas: memória de entrada, kernel e memória de saída. Nesse contexto, também foi inicializada a memória do golden model. É relevante ressaltar que, com exceção da memória de saída, todas as outras foram inicializadas pelo arquivo de texto gerado com Python, que produziu um arquivo txt com a matriz em forma de coluna. Para assegurar a confiabilidade do teste, um golden model específico foi desenvolvido. Esse modelo utilizava uma lógica interna para ler o arquivo de texto contendo a matriz da imagem gerada pelo script Python. Posteriormente, o modelo foi integrado ao testbench, permitindo a comparação direta entre a saída gerada pelo circuito e os dados armazenados no golden model. Durante o processamento, o sistema realizou operações conforme programado e armazenou os resultados na memória de saída. Ao final desse processo, uma etapa crucial de verificação ocorreu. Cada valor na memória de saída, representando o resultado do processamento, foi comparado com a correspondente posição na memória do golden model.

III. RESULTADOS

Através do circuito pronto, a imagem 5 foi utilizada para realizar a aplicação do kernel. Os resultados obtidos podem ser visualizados na imagem 6.

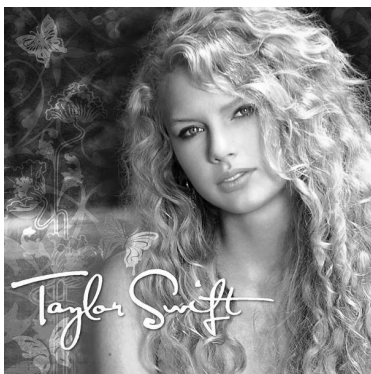


Fig. 5. Imagem modelo antes de aplicar o filtro gaussiano 3x3.
Fonte: Universal Music

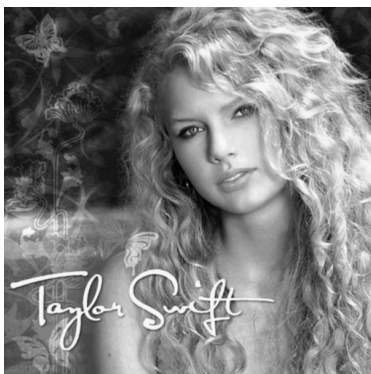


Fig. 6. Resultado após aplicação do filtro gaussiano 3x3

O resultado obtido é melhor visualizado quando aplicamos um zoom (demonstração nas imagens 7 e 8), ou quando o kernel é aplicado mais vezes. Contudo, apesar de sutil, os resultados obtidos foram bastante satisfatórios.



Fig. 7. Recorte da imagem original



Fig. 8. Recorte da imagem após aplicação do filtro

Os resultados de síntese obtidos para o circuito foram de: clock = 10ns, tempo de execução = 180ms. Os outros dados obtidos podem ser visualizados na imagem 9

Flow Status	Successful - Wed Nov 29 23:10:21 2023
Quartus II 64-bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	topo
Top-level Entity Name	topo
Family	Cyclone IV GX
Total logic elements	339 / 21,280 (2 %)
Total combinational functions	336 / 21,280 (2 %)
Dedicated logic registers	87 / 21,280 (< 1 %)
Total registers	87
Total pins	73 / 167 (44 %)
Total virtual pins	0
Total memory bits	0 / 774,144 (0 %)
Embedded Multiplier 9-bit elements	0 / 80 (0 %)
Total GXB Receiver Channel PCS	0 / 4 (0 %)
Total GXB Receiver Channel PMA	0 / 4 (0 %)
Total GXB Transmitter Channel PCS	0 / 4 (0 %)
Total GXB Transmitter Channel PMA	0 / 4 (0 %)
Total PLLs	0 / 4 (0 %)
Device	EP4CGX22CF19C6
Timing Models	Final

Fig. 9. Dados de síntese do circuito

REFERÊNCIAS

- [1] Wikipedia contributors. (2023, October 17). Kernel (image processing). In Wikipedia, The Free Encyclopedia. Retrieved 01:51, November 29, 2023
- [2] Lucidchart criado pelo grupo com os diagramas do projeto